

Case Study: Manual API Testing — FakeStore API

(<https://fakestoreapi.com/>)

Title:

Testing the “ShopEase” e-Commerce API (using FakeStoreAPI)

Background / Context:

ShopEase is a fictional e-commerce frontend which uses **FakeStoreAPI** (a free public REST API) as its backend data source. You (as a tester) don’t control the backend, but your job is to manually test the API endpoints to ensure they behave correctly (for expected and unexpected use cases).

FakeStoreAPI offers endpoints for products, categories, users, carts, and authentication.

Available Endpoints:

Here are some of the key endpoints you should test:

HTTP Method	Endpoint	Description
GET	/products	Returns list of all products
GET	/products/{id}	Returns a single product by ID
GET	/products/categories	Returns list of all product categories
GET	/products/category/{categoryName}	Returns products within that category
POST	/carts	Creates or updates a shopping cart (simulate adding items)
GET	/carts/{cartId}	Get details of a specific cart / user's cart
GET	/users	Returns list of users
GET	/users/{id}	Returns a specific user by ID
POST	/auth/login	Authenticate a user and get a token

Note:

- FakeStoreAPI supports standard HTTP methods (GET, POST, PUT, DELETE, etc.).
- Some endpoints are read-only (i.e. existing public mock data).
- The API returns JSON responses.
- You may combine endpoints (for example, fetch a product from /products, then use that ID to request /products/{id}).

Problem Statement:

You are the manual API tester for ShopEase. Your task is to thoroughly test the FakeStoreAPI endpoints to ensure correct behavior under various scenarios.

Your tasks:

1. List out Test Scenarios / Test Cases

- For each endpoint above, prepare both **positive** and **negative** test cases.
- Define input (URL + parameters / body), expected status code, and expected response (fields, types).
- Examples of negative cases: non-existing IDs, invalid category names, malformed request body, missing fields, etc.

2. Execute Tests Using Postman

- Hit each endpoint with your test cases.
- Observe the actual status codes, response payloads, headers, error messages (if any).
- Capture responses (screenshot or copy) for document.

3. Validate Response Schema & Data

- Check JSON fields and types (for example, in /products, each item must have id, title, price, description, category, image, maybe rating).
- For list endpoints, check array size, pagination (if supported), filtering (if query params exist).

4. Chaining / Dependent Tests

- For example: pick a product ID from /products, then request /products/{id}.
- Use the categories list to test /products/category/{categoryName}.
- Simulate adding items to a cart via /carts and then retrieve it back with /carts/{id}.

5. Bug / Issue Reporting

- List any discrepancies: mismatched status codes, missing fields, inconsistent data, unhandled error cases, etc.
- Document them in a bug log (ID, description, steps to reproduce, actual vs expected, severity).

6. Suggestion / Improvement

- o Based on your findings, suggest improvements (better error messages, stricter validation, better status codes, etc.).

Deliverables:

- **Test Case Document / Spreadsheet** (columns like Test ID, Endpoint, Input, Expected, Actual, Pass/Fail, Notes)
- **Test Execution Report** (summary, key observations, screenshots, sample JSON responses)
- **Bug / Issues Log**